

Task to be performed

Project Overview:

In this project, you will build a Conditional Generative Adversarial Network (cGAN) to generate images of cats or dogs, conditioned on class labels. By integrating label information directly into both the generator and discriminator networks, you will learn how to control the category of generated outputs.

This project involves loading and preprocessing the Cats vs Dogs dataset, implementing a cGAN architecture, training the model, and visualizing class-conditioned outputs.

Dataset Description

The **Cats vs Dogs** dataset contains over 23,000 labeled images. For this project:

- Label 0 = Cat
- Label 1 = Dog

You will use unpaired, labeled images to train your cGAN model. The dataset can be directly loaded using `tensorflow_datasets`.

Project Tasks

Part 1: Data Preparation

1. Dataset Loading

- Use TensorFlow Datasets (`cats_vs_dogs`)

use below code snippet to load it-

```
import tensorflow as tf
import tensorflow_datasets as tfds
import matplotlib.pyplot as plt
import numpy as np
from tensorflow.keras import layers
```

```
(ds_train, _), ds_info = tfds.load(
    'cats_vs_dogs',
    split=['train[:90%]', 'train[90%:]'],
    shuffle_files=True,
    as_supervised=True,
    with_info=True
)
```

- Split into training and testing subsets

2. Preprocessing

- Resize all images to 64×64 pixels
- Normalize pixel values to the `[-1, 1]` range
- Cast labels to integer format
- Create training batches with labels

Part 2: Conditional GAN Architecture

1. Generator

- Accepts both noise vector and class label as input
- Uses label embedding and concatenates it with noise
- Builds up to a $64 \times 64 \times 3$ image using Conv2DTranspose layers

2. Discriminator

- Accepts both real/fake image and class label
- Embeds label and concatenates it with image
- Uses Conv2D layers to classify input as real/fake

Part 3: Loss Functions and Optimizer

- **Generator Loss:** Binary cross-entropy encouraging discriminator to classify generated images as real
- **Discriminator Loss:** Binary cross-entropy distinguishing real vs fake images
- **Optimizers:** Adam optimizer for both networks

Part 4: Training the Model

- Train the generator and discriminator alternately
- Use `@tf.function` for optimized performance
- Train for a minimum of 10 epochs
- Print training progress per epoch

Part 5: Evaluation and Visualization

- Generate images conditioned on randomly chosen labels
- Display generated images with corresponding label titles
- Ensure images are normalized back to $[0, 1]$ for display

Submission Requirements

- Python script or Jupyter notebook with:
 - Code for model architecture, training loop, and visualization
 - Comments explaining each step
- Visual outputs showcasing label-conditioned generation